Kernel Density Estimation using R

Jyotishka Datta

2025-11-17

Density Estimation

Suppose a random sample $X_1, \ldots, X_n$ is drawn from a population with an unknown continuous probability density function $f(x)$.

Goal: estimate $f(x)$.

Histogram

Partition the range of data into subintervals

$$a_1 < a_2 < \cdots < a_k.$$

For $x \in (a_i, a_{i+1}]$,

$$\hat{f}(x) = \frac{\#\{\text{observations} \in (a_i, a_{i+1}]\}}{n(a_{i+1} - a_i)}.$$

Bandwidth

- Typically, the intervals have equal length h: $(a_{i+1} - a_i) = 2h$ for $i = 1, \ldots, k$. The equal interval length h is often referred to as the **bandwidth**.
- In this case, for any x within the data range,

$$\hat{f}(x) = \frac{\#\{\text{observations within } h \text{ of } x\}}{2nh} = \frac{1}{2nh} \sum_{i=1}^{n} I(|X_i - x| \leq h).$$

- Narrow intervals (smaller h): histogram has a choppy appearance, large variance.
- Large intervals (larger h): histogram may lose local features, large bias.
- Generally, a larger sample size n requires a smaller h, thus more intervals.

Suggested bandwidth:

$$h = \frac{1.75}{n^{1/3}} S,$$

where S is the sample standard deviation.

Histograms

- Histograms provide an idea about the data distribution.
- Look at the definition again:

$$\hat{f}(x) = \frac{\text{No. of observations within } h \text{ of } x}{n \times 2h}$$

or,

$$\hat{f}(x) = \frac{1}{2nh} \sum_{i=1}^{n} I(|X_i - x| \leq h).$$

- Narrow intervals (smaller h): histogram has a choppy appearance, large variance.
- Large intervals (larger h): histogram may lose the local feature, large bias.

R `hist()`

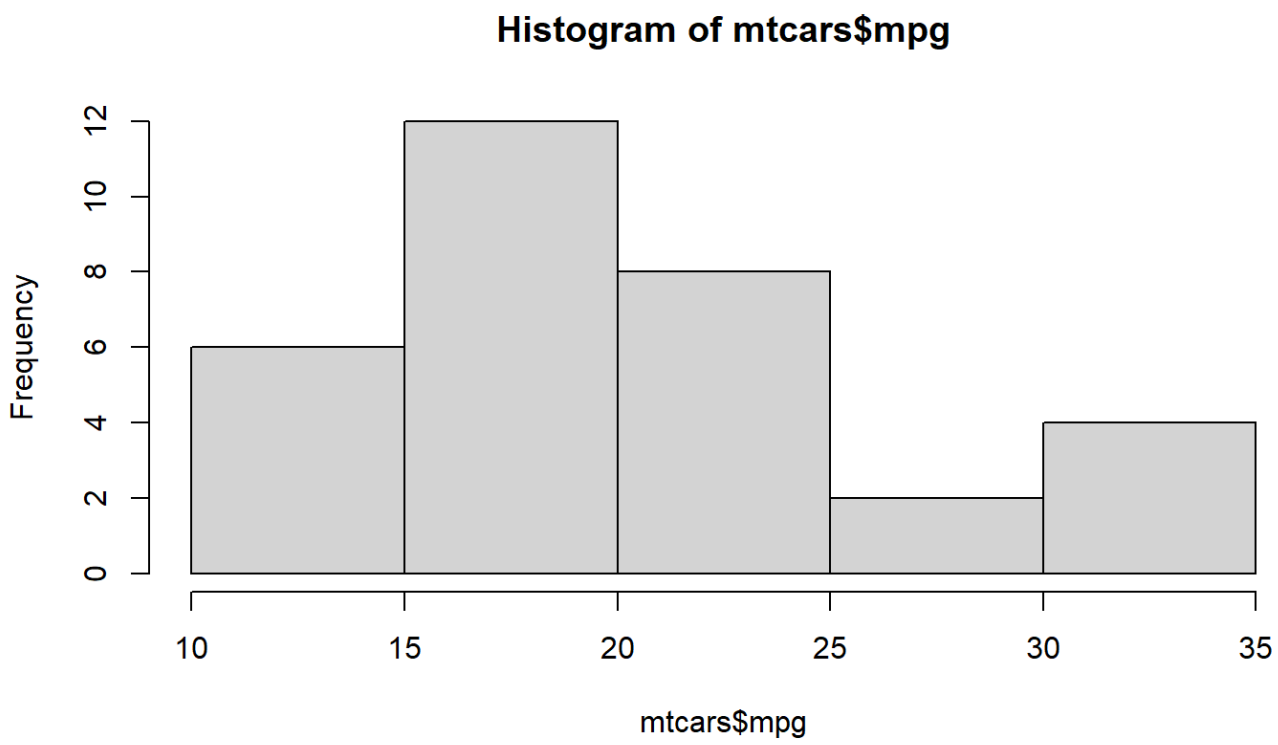
- In base R's `hist()` method, you can use the `break` argument:
- `breaks` can be one of:

1. a vector giving the breakpoints between histogram cells,
2. a function to compute the vector of breakpoints,
3. a single number giving the number of cells for the histogram,
4. a character string naming an algorithm to compute the number of cells (see 'Details'),
5. a function to compute the number of cells.

Sampling distribution

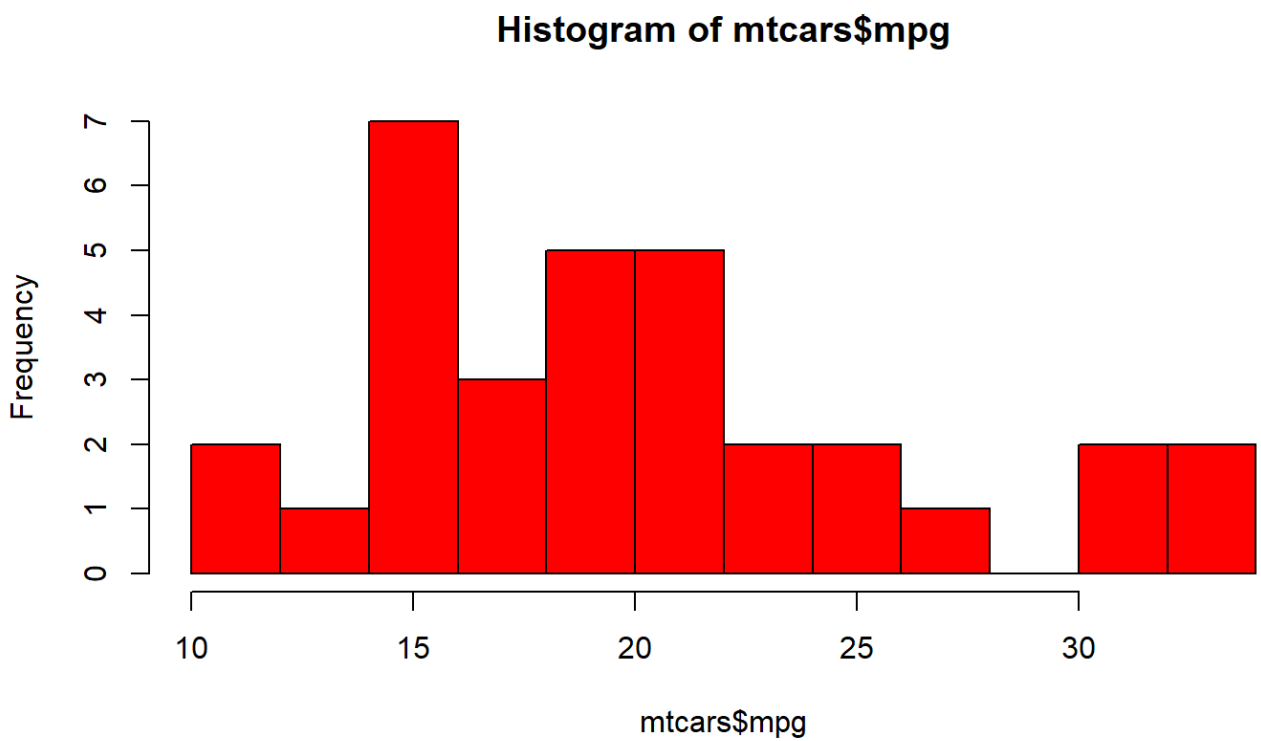
- By default, number of breaks is calculated by using Sturges' formula.

```
data(mtcars)
hist(mtcars$mpg)
```



Colored Histogram with Different Number of Bins

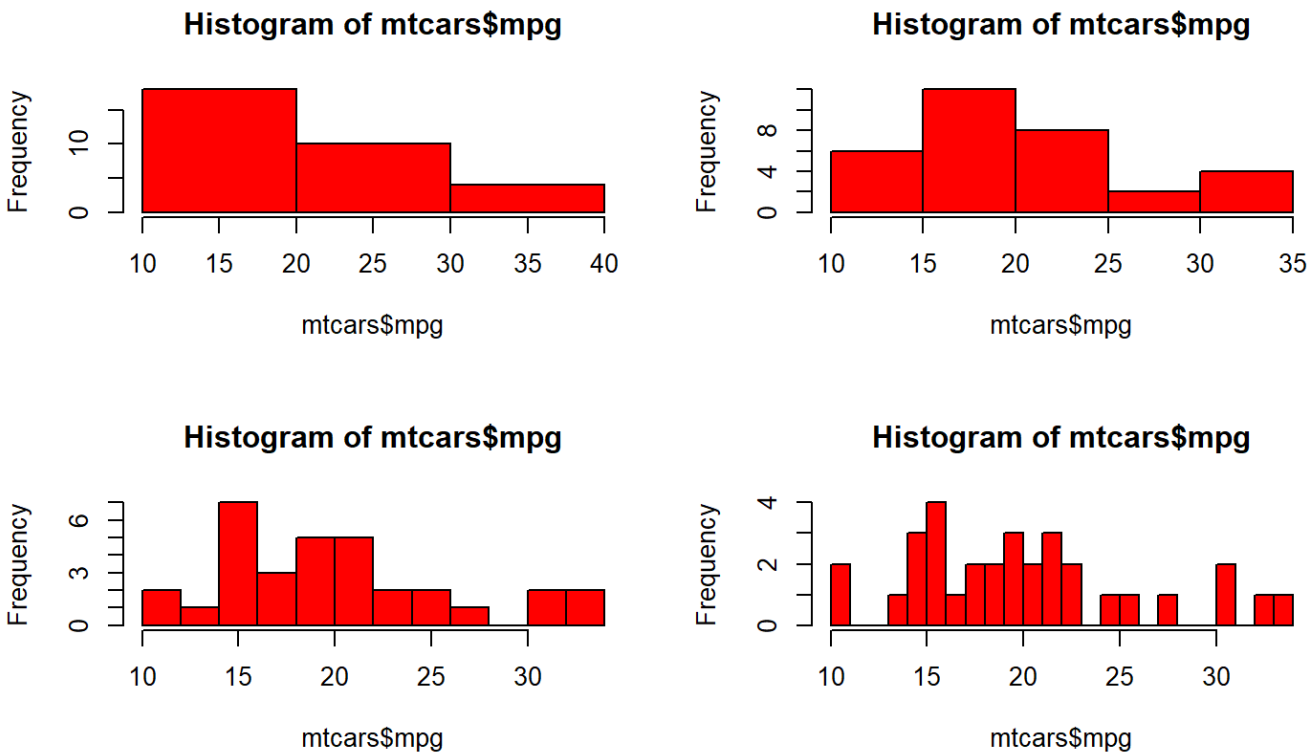
```
hist(mtcars$mpg, breaks=12, col="red")
```



Different h

```
par(mfrow = c(2,2))
hist(mtcars$mpg, breaks=2, col="red")
hist(mtcars$mpg, breaks=8, col="red")
hist(mtcars$mpg, breaks=12, col="red")
hist(mtcars$mpg, breaks=20, col="red")
par(mfrow = c(1,1))
```

Different h


Histogram of mtcars$mpg
Frequency
0
10
10
15
20
25
30
35
40
mtcars$mpg
Histogram of mtcars$mpg
Frequency
0
4
8
10
15
20
25
30
35
mtcars$mpg
Histogram of mtcars$mpg
Frequency
0
3
6
10
15
20
25
30
mtcars$mpg
Histogram of mtcars$mpg
Frequency
0
2
4
10
15
20
25
30
mtcars$mpg

Kernel Density Estimation

- Drawback of histogram: density estimation is piecewise constant and thus unsmooth.
- Main idea of kernel density estimation: instead of counting number of observations within h of x, we take a certain weighted average of data points near x to estimate $f(x)$.
- Usually we assign higher weights to data points closer to x, and lower weights to those further away from x.
- Kernel Density Estimate:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{|X_i - x|}{h}\right).$$

where $K(\cdot)$ is a Kernel function.

- Choices: h bandwidth and $K()$ the kernel.

Choices of kernel function

- Uniform: $K(u) = \frac{1}{2} I(|u| \leq 1)$. Reduces to histogram.
- Gaussian: $K(u) = \phi(u)$ (standard normal density).
- Triangle: $K(u) = (1 - |u|) I(|u| \leq 1)$.
- Epanechnikov: $K(u) = \frac{3}{4}(1 - u^2) I(|u| \leq 1)$.
- Biweight: $K(u) = \frac{15}{16}(1 - u^2)^2 I(|u| \leq 1)$.
- Triweight: $K(u) = \frac{35}{32}(1 - u^2)^3 I(|u| \leq 1)$.
- Minimum variance kernel (allowing negative weight):

$$K(u) = \frac{1}{8}(3 - 5u^2) I(|u| \leq 1).$$

Choice of bandwidth h

A rule of thumb (Harfle, 1991):

$$h = \frac{1.06}{n^{1/5}} S,$$

where S is the sample standard deviation, or a more robust version such as

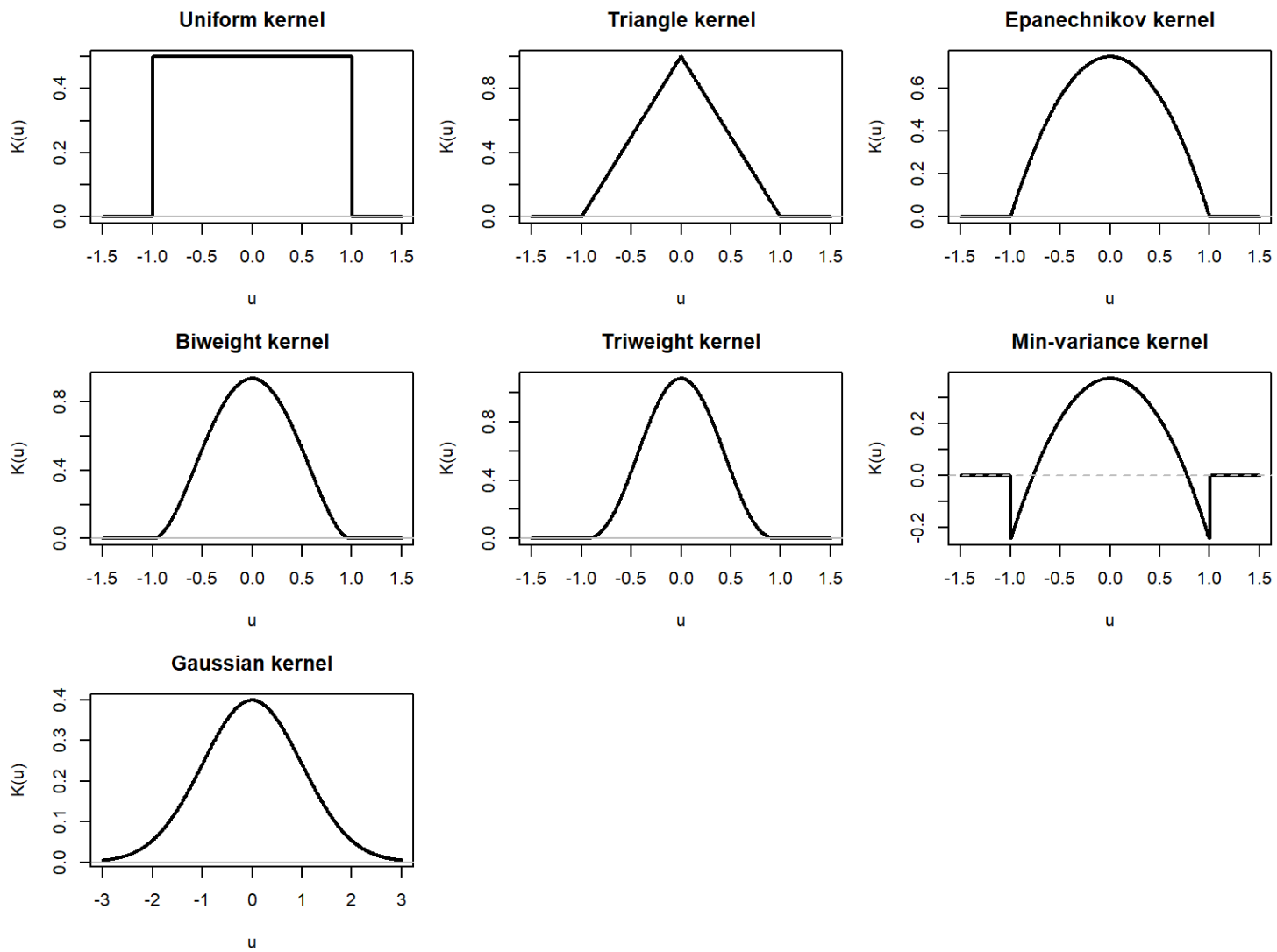
- scaled interquartile range:

$$S = \frac{X_{(0.75)} - X_{(0.25)}}{1.34},$$

- scaled MAD (median absolute deviation):

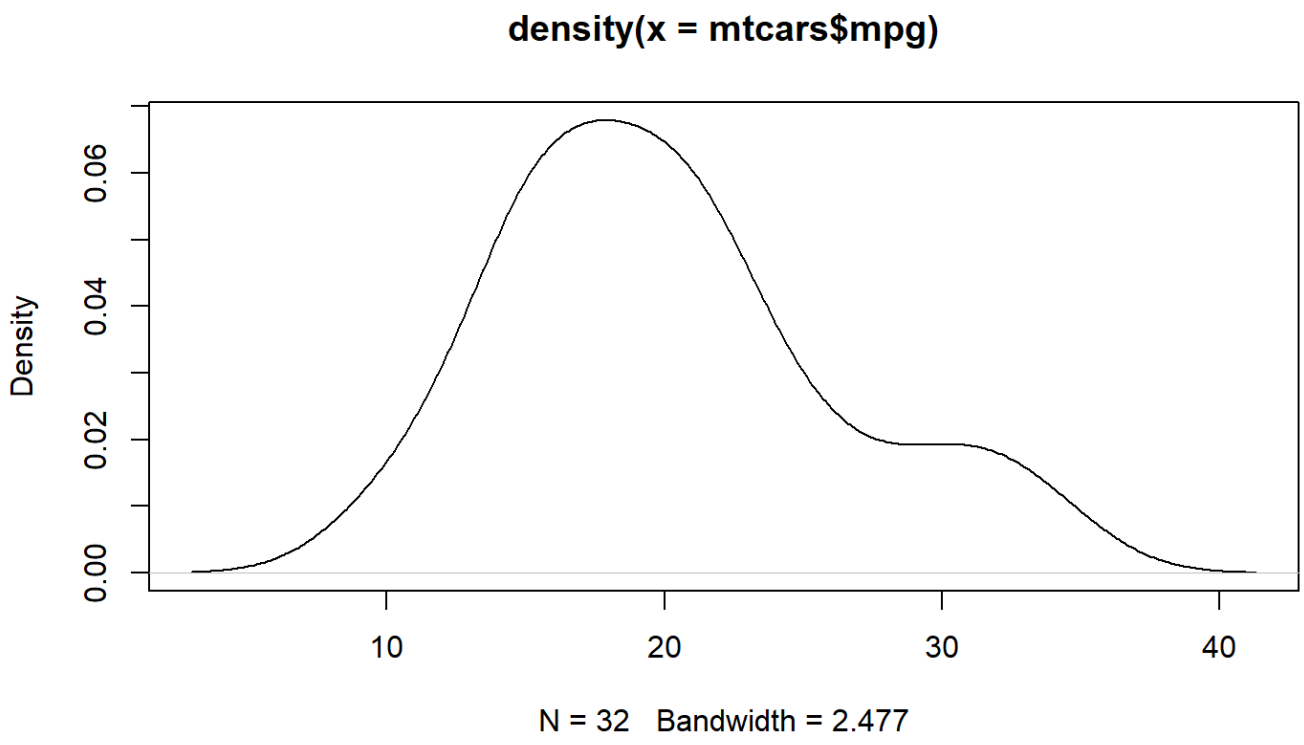
$$S = 1.4826 \cdot \mathrm{Median}\{|X_i - \mathrm{median}(X_i)|\}.$$

Kernel function shapes



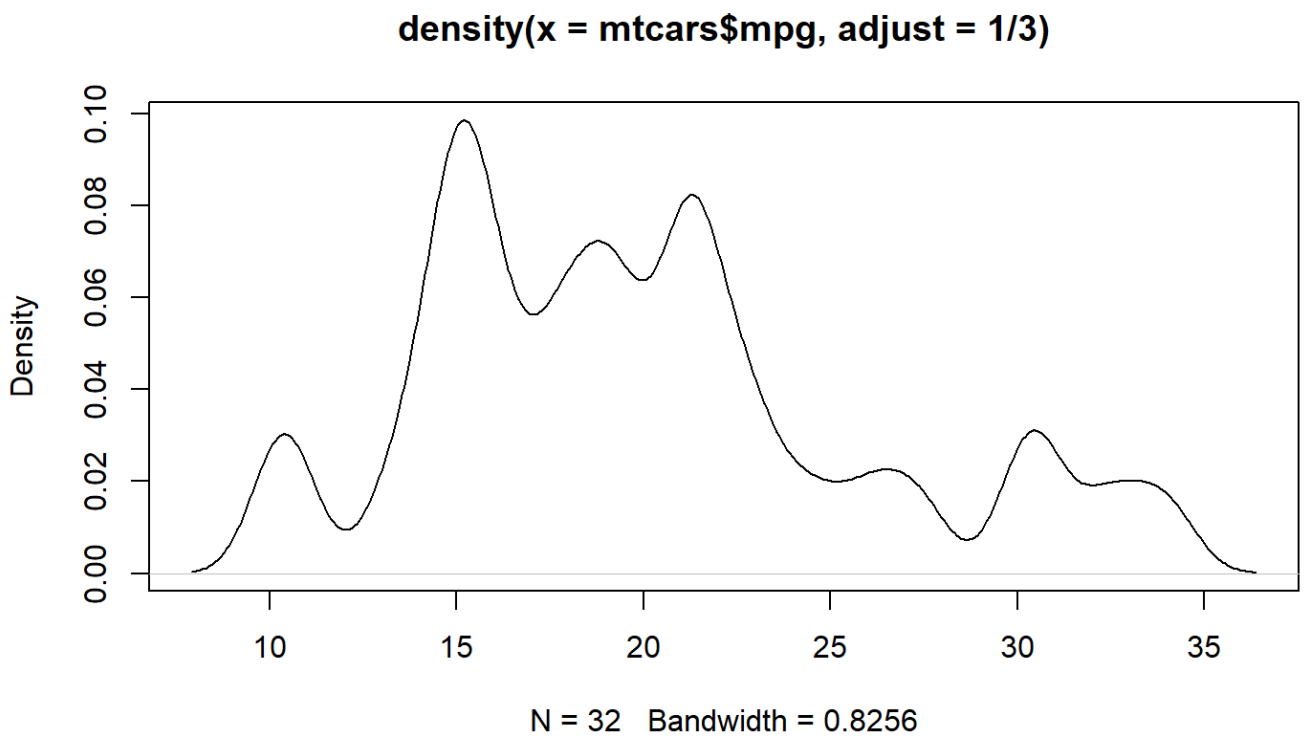
Kernel Density Plot

```
d <- density(mtcars$mpg) # returns the density data
plot(d) # plots the results
```



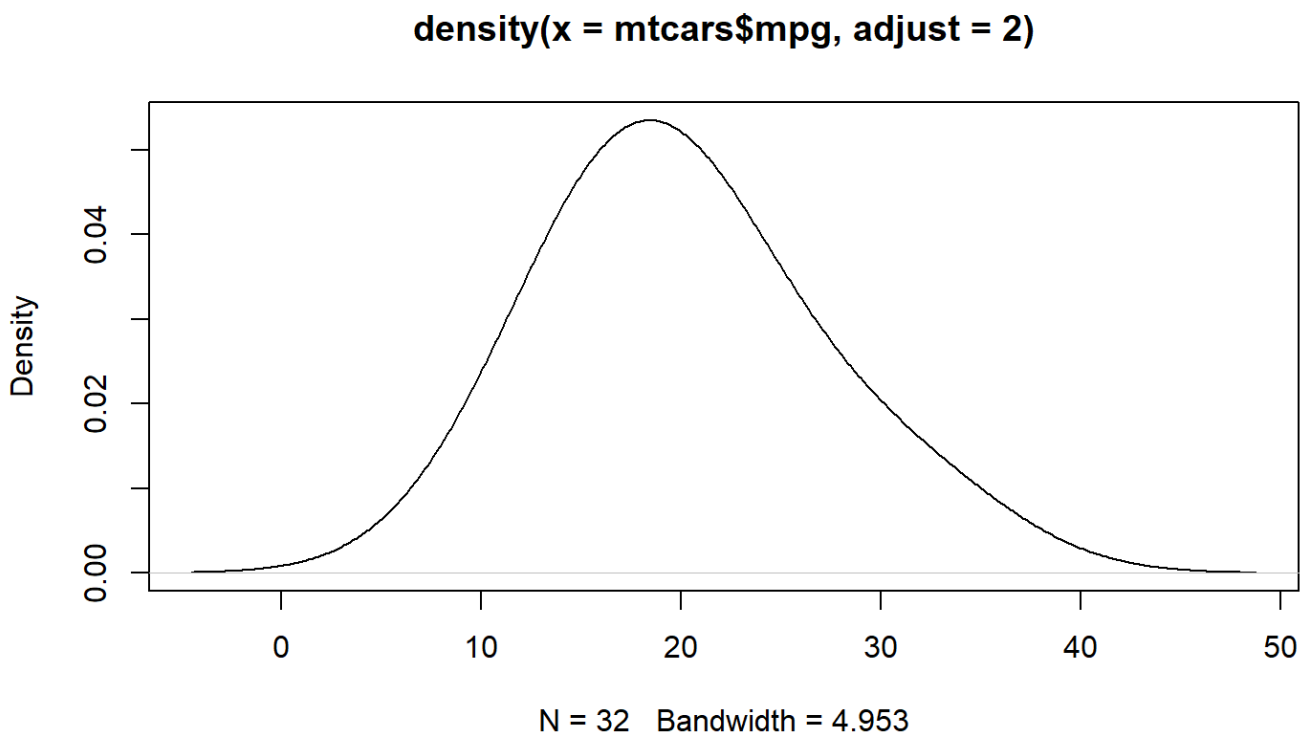
Kernel Density Plot with 1/3rd bandwith

```
d <- density(mtcars$mpg,adjust = 1/3) # returns the density data
plot(d) # plots the results
```



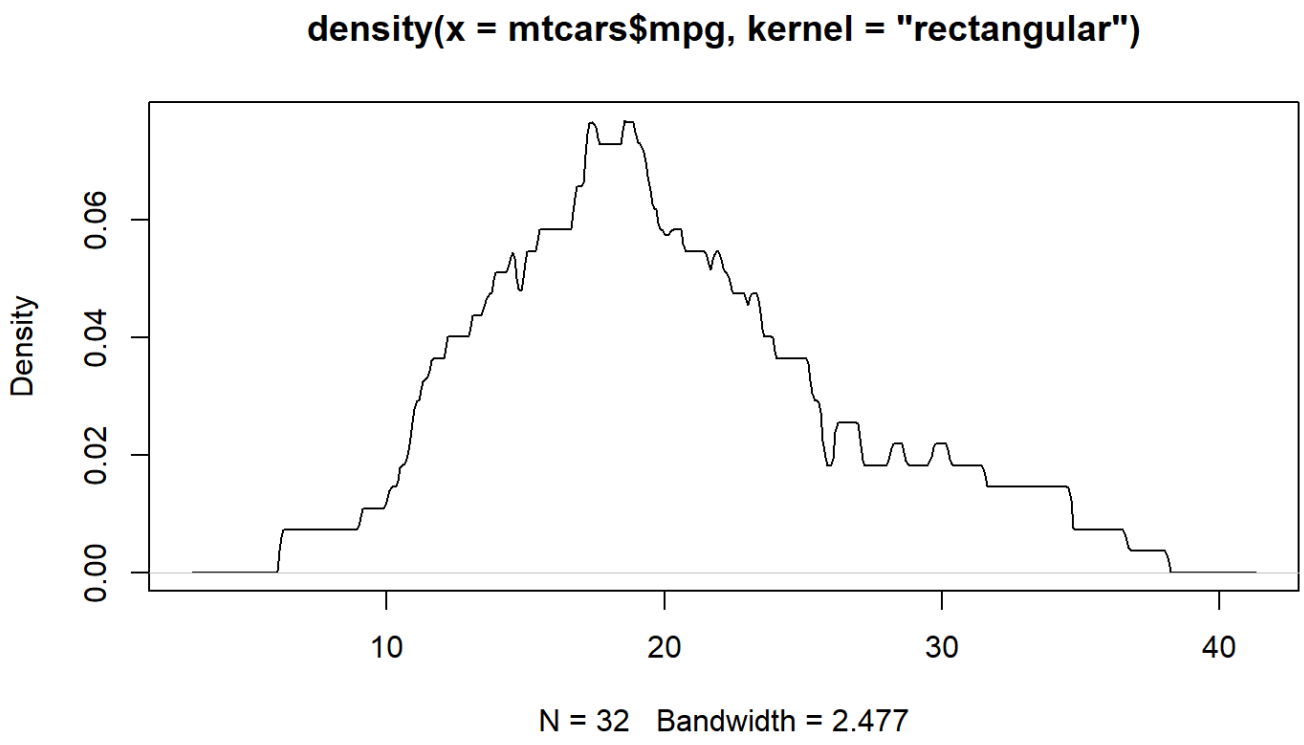
Kernel Density Plot with 2x bandwith

```
d <- density(mtcars$mpg,adjust = 2) # returns the density data
plot(d) # plots the results
```



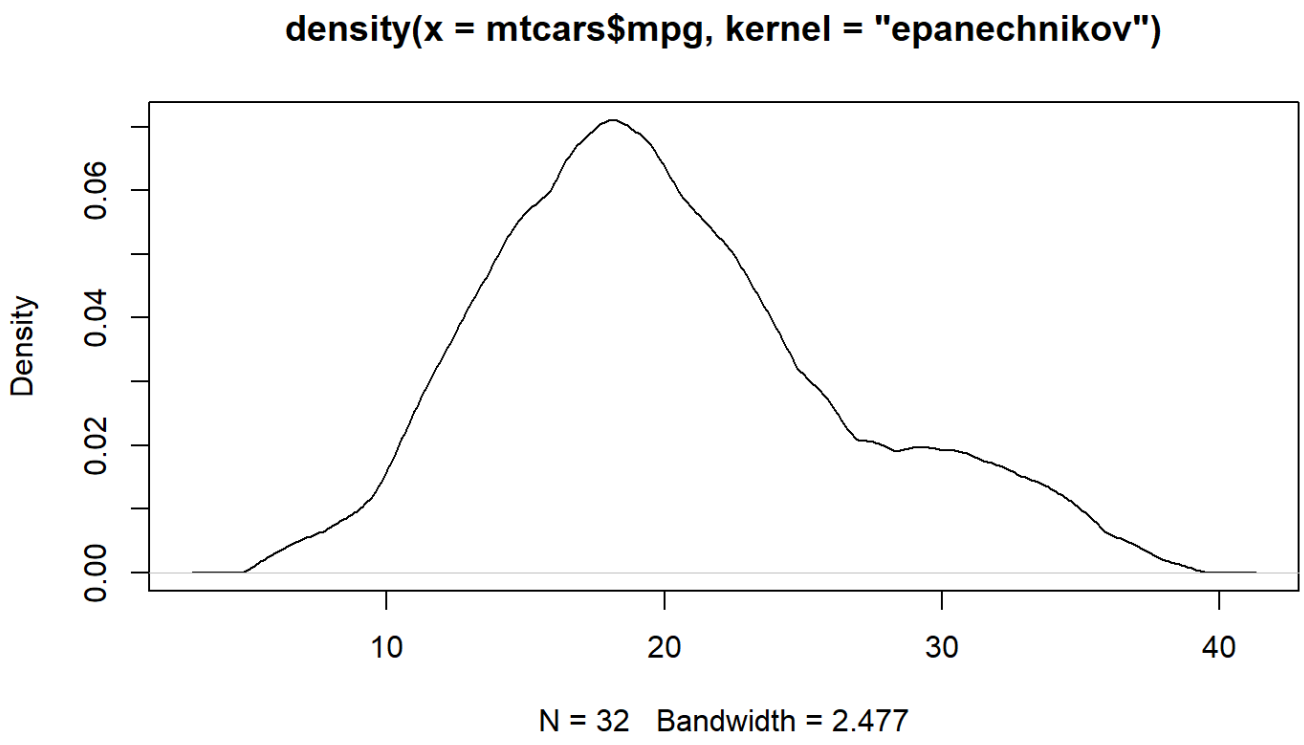
Kernel Density Plot with a rectangular kernel

```
d <- density(mtcars$mpg, kernel = "rectangular") # returns the density data
plot(d) # plots the results
```



Kernel Density Plot with the Epanechnikov kernel

```
d <- density(mtcars$mpg, kernel = "epanechnikov") # returns the density data
plot(d) # plots the results
```



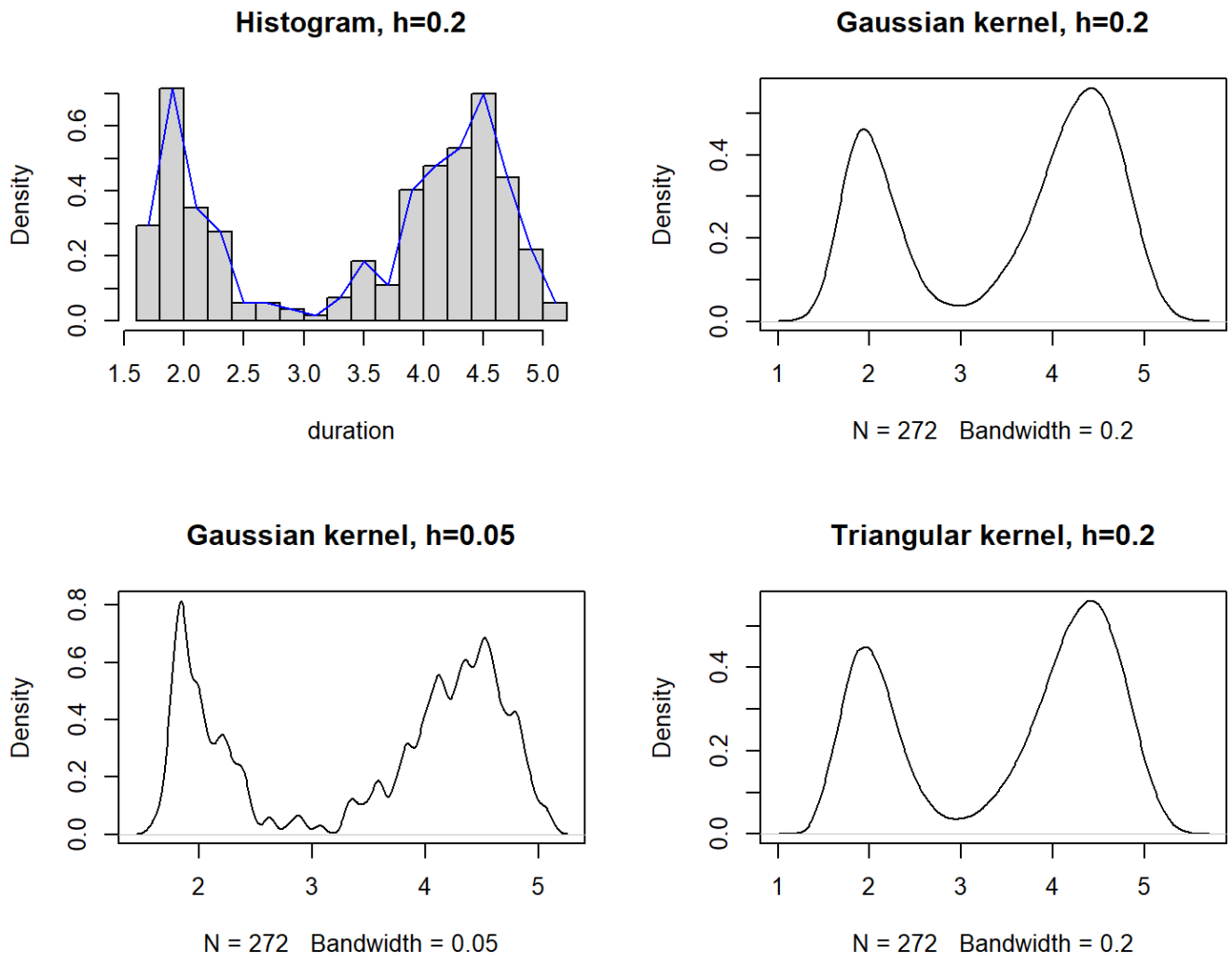
Faithful Geyser

```
#take a look at the data set (the first 5 observations)
faithful[1:5,]
```

```
##   eruptions waiting
## 1     3.600      79
## 2     1.800      54
## 3     3.333      74
## 4     2.283      62
## 5     4.533      85
```

```
duration = faithful$eruptions
n=length(duration)
#estimate the density using histogram and kernel estimation
```

Different Kernels

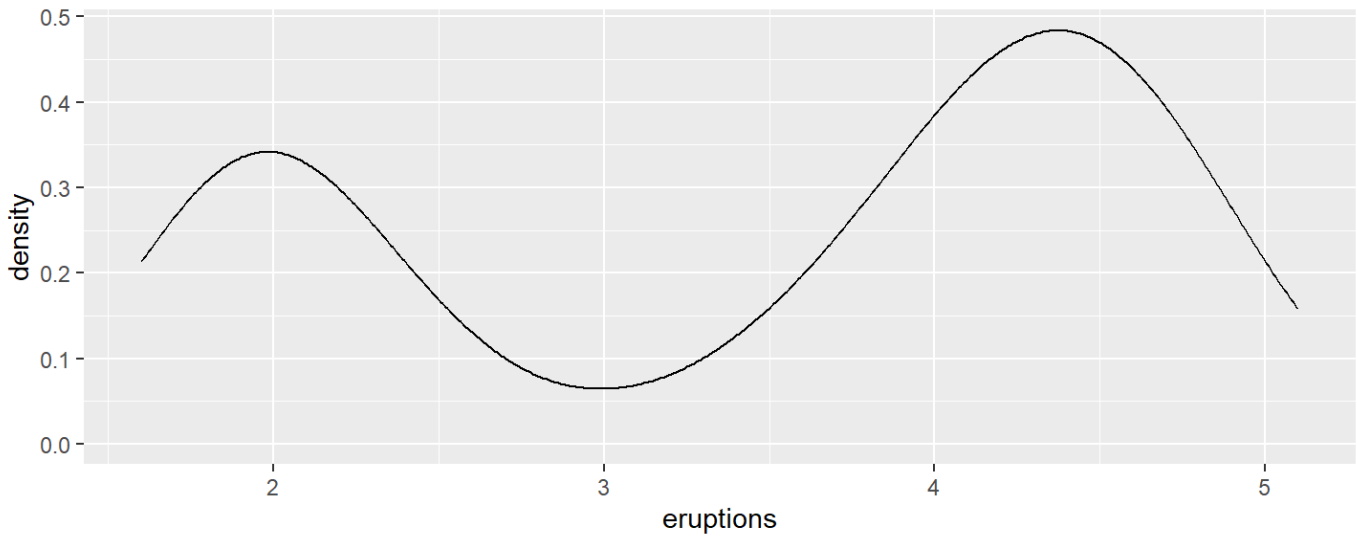


We can also use ggplot

```
library(ggplot2)
```

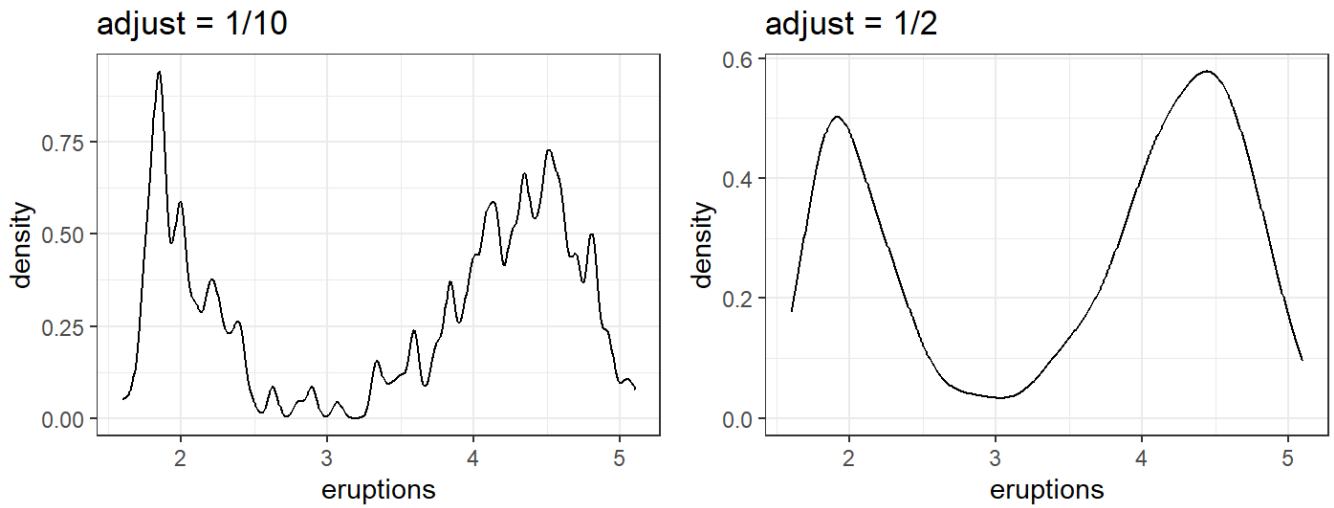
```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
ggplot(data = faithful, aes(x = eruptions)) + geom_density()
```



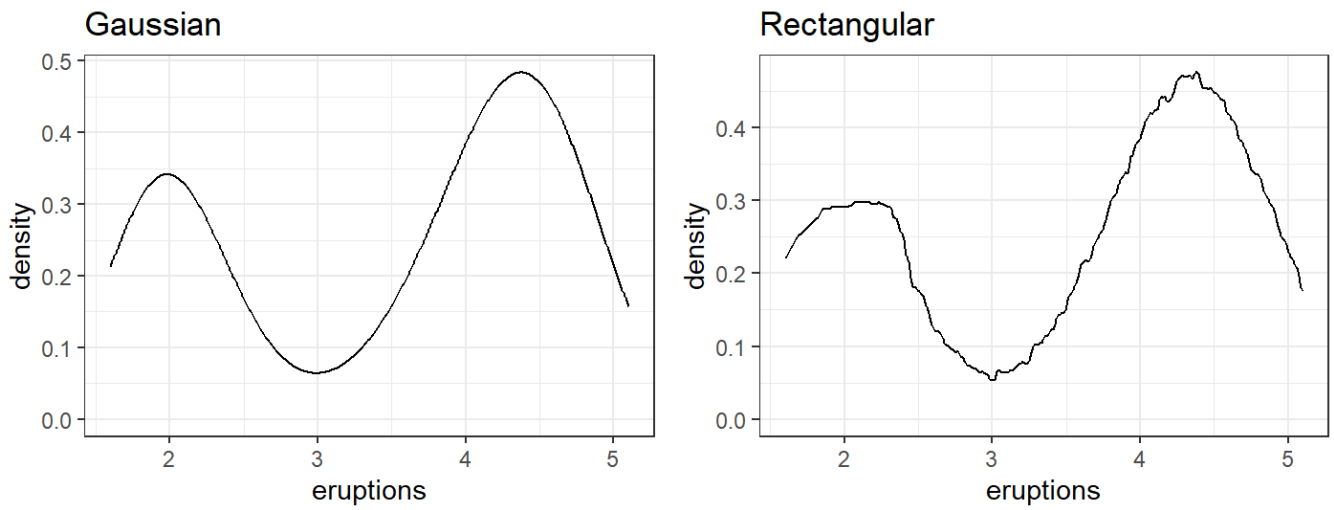
Different bandwidth

```
library(patchwork)
p1 <- ggplot(data = faithful, aes(x = eruptions)) + geom_density(adjust = 1/10) +
  labs(title = "adjust = 1/10")+theme_bw()
p2 <- ggplot(data = faithful, aes(x = eruptions)) + geom_density(adjust = 1/2) +
  labs(title = "adjust = 1/2")+theme_bw()
p1+p2
```



Different Kernels

```
p1 <- ggplot(data = faithful, aes(x = eruptions)) + geom_density(kernel = "gaussian") +
  labs(title = "Gaussian")+theme_bw()
p2 <- ggplot(data = faithful, aes(x = eruptions)) + geom_density(kernel = "rectangular") +
  labs(title = "Rectangular")+theme_bw()
p1+p2
```



Drawing KDE manually

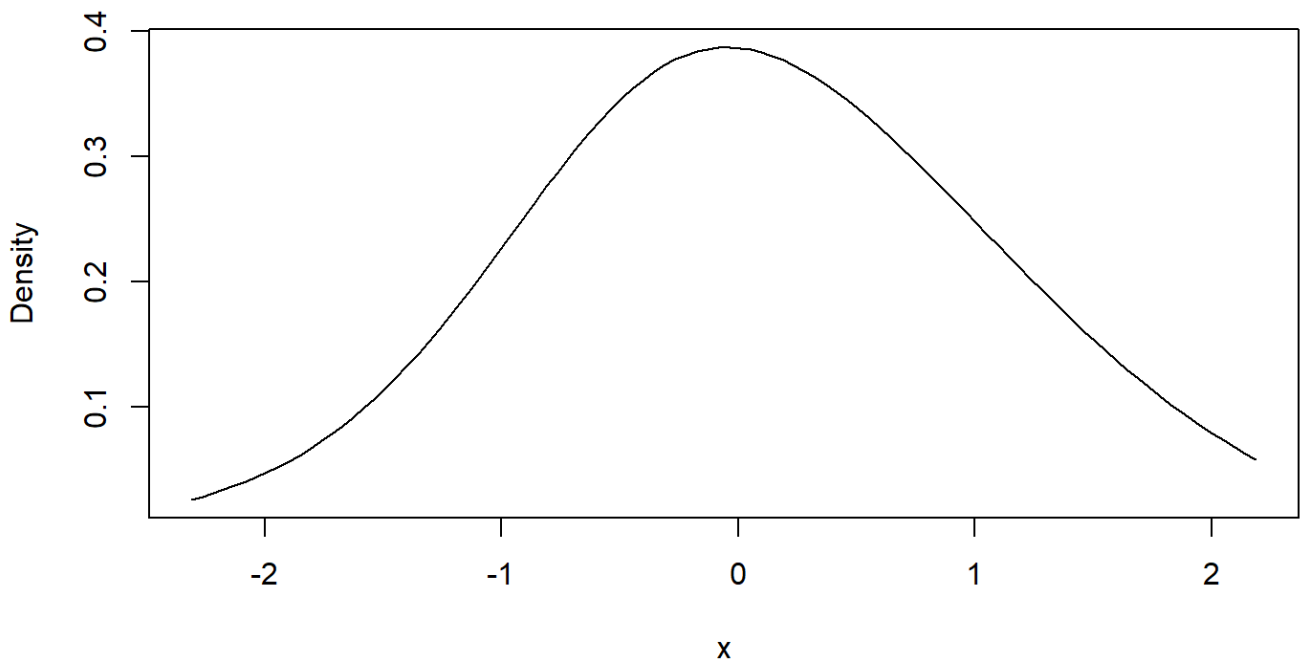
- We choose a range of values to plot using the minimum and maximum values of the data, and create a sequence of values using the seq() function. We set the bandwidth for the kernel density estimation using the bw variable.
- Next, we use the sapply() function to compute the kernel density estimate for each value in the sequence.

```
# Generate some sample data
set.seed(123)
x <- rnorm(100)

# Choose a range of values to plot
xmin <- min(x)
xmax <- max(x)
xseq <- seq(xmin, xmax, length.out = 100)

# Set the bandwidth for the kernel density estimation
bw <- 0.5

# Compute the kernel density estimate
kde <- sapply(xseq, function(xi) {
  mean(dnorm((xi - x)/bw)/bw)
})
```

Drawing KDE manually

- The R code on the last slide would produce this KDE.